



Guia rápido dos comandos essenciais do terminal (Linux / Git Bash)

O **terminal** é uma janela onde você digita comandos em texto para controlar o computador. Ele permite executar ações sem usar o mouse — o que torna o trabalho mais rápido, preciso e automatizável.

O terminal faz parte de um tipo de interface chamada **CLI** (*Command Line Interface*), ou **interface de linha de comando**.

Enquanto interfaces gráficas (GUI) usam janelas, botões e ícones, a CLI usa **texto** — e por isso é mais leve e poderosa para tarefas técnicas.

Dentro do terminal, quem realmente interpreta e executa os comandos é o **shell**.

O shell é o “cérebro” por trás da CLI: ele lê o que você digitou, entende o comando e conversa com o sistema operacional para realizar a ação.

Existem vários tipos de shells, cada um com seus comandos e recursos:

- **Bash, Zsh, Fish** → shells de sistemas Linux/Unix
- **PowerShell** → shell moderno do Windows
- **CMD** → shell antigo do Windows
- **SQL Shell** → interface de comandos para bancos de dados
- **Python Shell / REPL** → interface interativa para rodar comandos Python

Em todos os casos, o shell é a camada que recebe comandos e devolve resultados.

Este guia explica os comandos fundamentais de **navegação, arquivos, pastas e permissões**, com exemplos reais de saída e analogias simples.

Funciona tanto em **Linux** quanto em **Git Bash no Windows**.



1. Onde estou? — **pwd**

```
pwd
```

Significado: **pwd** vem de **print working directory**

→ literalmente: “imprimir o diretório de trabalho”.

Função: mostra o caminho completo (absoluto) da pasta atual.

Use quando quiser confirmar **em qual pasta você está** antes de rodar outros comandos.

Exemplo de saída:

```
/c/Users/ander/OneDrive/Área de Trabalho
```

Interpretação:

- `/c/Users/...` → caminho no estilo Linux usado pelo Git Bash
- você está dentro da pasta **Área de Trabalho**
- esse é o caminho **completo**, desde a raiz do sistema

Analogia: pense no `pwd` como o “você está aqui” do mapa do terminal.

2. Navegar entre pastas — `cd`

O comando `cd` significa **change directory**
→ literalmente: “mudar de diretório”.

Ele serve para **entrar, sair e mover-se entre pastas** no terminal.

Entrar em uma pasta

```
cd projetos
```

Entra na pasta `projetos` que está dentro da pasta atual.

Voltar uma pasta

```
cd ..
```

`..` significa “pasta acima”.

Use para subir um nível na hierarquia.

Ir para a home

```
cd ~
```

`~` representa sua **pasta pessoal** (home).

No Git Bash, geralmente é algo como:

```
/c/Users/seu-usuario
```

Ir para a raiz do sistema

```
cd /
```

/ é a **raiz** do sistema de arquivos — o ponto mais alto da árvore.

Símbolos importantes

- ~ → home (pasta do usuário)
- . → diretório atual
- .. → diretório acima

Analogia: pense no `cd` como andar entre pastas como se fossem salas de um prédio — `..` sobe um andar, `.` é a sala onde você já está, `~` é sua casa.

3. Listar arquivos e pastas — `ls`

O comando `ls` significa **list**

→ literalmente: “*listar*” os arquivos e pastas do diretório atual.

Listar conteúdo

```
ls
```

Mostra apenas os nomes dos arquivos e pastas.

Listar com detalhes

```
ls -l
```

`-l` significa **long format** (formato longo).

Mostra permissões, dono, tamanho e data de modificação.

Exemplo:

```
-rw-r--r-- 1 anderson users 1024 Jun 12 09:00 notas.txt
drwxr-xr-x 2 anderson users 4096 Jun 12 08:50 projetos
```

Como interpretar:

- `-rw-r--r--` → permissões do arquivo
- `d` no início → é um diretório (`projetos`)
- `anderson` → dono
- `1024 / 4096` → tamanho em bytes
- `Jun 12 09:00` → última modificação

Listar arquivos ocultos

```
ls -a
```

`-a` significa **all** → mostra *todos* os arquivos, incluindo os ocultos (que começam com `.`).

Combinar opções

```
ls -la
```

Lista **tudo** com **detalhes**.

Analogia: pense no `ls` como abrir uma pasta no explorador de arquivos — só que em modo texto.

4. Criar pastas — `mkdir`

O comando `mkdir` significa **make directory**

→ literalmente: “*criar diretório*” (diretório = pasta).

Use quando quiser criar novas pastas no sistema.

Criar uma pasta

```
mkdir nova_pasta
```

Cria uma pasta chamada `nova_pasta` dentro do diretório atual.

Criar várias pastas de uma vez

```
mkdir pasta1 pasta2 pasta3
```

Cria todas as pastas listadas em um único comando.

Dica: nomes com espaço precisam de aspas, por exemplo:

```
mkdir "Minhas Fotos"
```

Analogia: pense no `mkdir` como adicionar novas “caixas” dentro da sua estante de arquivos.

5. Criar arquivos — `touch`

O comando `touch` significa **tocar** (no sentido de “encostar” em um arquivo).

No Unix, “tocar” um arquivo atualiza sua **data de modificação** — e, se ele **não existir**, o sistema o cria.

Criar um arquivo vazio

```
touch arquivo.txt
```

Se `arquivo.txt` não existir, ele será criado imediatamente.

Atualizar a data de modificação

```
touch arquivo.txt
```

Se o arquivo **já existir**, o conteúdo não muda — apenas a data/hora de modificação é atualizada.

Isso é útil para scripts, automações ou para marcar arquivos como “recentes”.

Dica: você pode criar vários arquivos de uma vez:

```
touch a.txt b.txt c.txt
```

Analogia: pense no **touch** como “dar um tapinha” no arquivo — se ele existe, você só o atualiza; se não existe, você o cria.



6. Apagar arquivos e pastas — **rm** e **rmdir**

O comando **rm** significa **remove**

→ literalmente: “*remover*” um arquivo ou pasta.

O comando **rmdir** significa **remove directory**

→ usado apenas para **pastas vazias**.

⚠ **Atenção:** no terminal, arquivos apagados **não vão para a lixeira**.
Depois do **rm**, não há “desfazer”.



Apagar um arquivo

```
rm arquivo.txt
```

Remove o arquivo definitivamente.



Apagar uma pasta com tudo dentro

```
rm -r nome_da_pasta
```

-r significa **recursive** → apaga a pasta e *todo o conteúdo* dentro dela.

Use com cuidado.



Apagar uma pasta vazia

```
rmdir pasta_vazia
```

Só funciona se a pasta estiver **completamente vazia**.

Dica de segurança:

Para confirmar antes de apagar cada item, use:

```
rm -ri nome_da_pasta
```

Analogia: `rm` é como um “delete permanente”; `rmdir` é como jogar fora uma caixa vazia.



7. Mover ou renomear — `mv`

O comando `mv` significa **move**

→ literalmente: “*mover*” um arquivo ou pasta.

Ele também serve para **renomear**, porque renomear nada mais é do que “mover” o arquivo para um novo nome.



Mover um arquivo para outra pasta

```
mv arquivo.txt pasta/
```

Move `arquivo.txt` para dentro da pasta `pasta/`.

Se a pasta não existir, o comando falha.



Renomear um arquivo

```
mv antigo.txt novo.txt
```

Troca o nome do arquivo sem alterar seu conteúdo.

Dicas úteis:

- Você pode mover **vários arquivos de uma vez**:

```
mv a.txt b.txt c.txt pasta/
```

- Para mover uma pasta inteira:

```
mv minha_pasta destino/
```

Analogia: `mv` funciona como arrastar um arquivo para outra pasta — ou simplesmente trocar o nome da caixa.



8. Copiar arquivos e pastas — `cp`

O comando `cp` significa **copy**

→ literalmente: “copiar” um arquivo ou diretório.

Ele cria **uma nova cópia**, mantendo o original intacto.



Copiar um arquivo

```
cp arquivo.txt copia.txt
```

Cria `copia.txt` com o mesmo conteúdo de `arquivo.txt`.

Se `copia.txt` já existir, será sobrescrito.



Copiar uma pasta inteira

```
cp -r pasta_original pasta_copia
```

`-r` significa **recursive** → copia a pasta e *todo o conteúdo* dentro dela (subpastas, arquivos, etc.).

Dicas úteis:

- Copiar vários arquivos para uma pasta:

```
cp a.txt b.txt c.txt destino/
```

- Copiar mantendo permissões e atributos:

```
cp -p arquivo.txt copia.txt
```

Analogia: `cp` funciona como “duplicar” um arquivo ou pasta — como copiar e colar no explorador gráfico.



9. Ver conteúdo de arquivos — `cat` e `less`

Mostrar conteúdo

```
cat arquivo.txt
```

Ver arquivo rolando

```
less arquivo.txt
```

Use **q** para sair.



10. Limpar a tela — **clear**

O comando **clear** significa **limpar** a tela do terminal.

Ele não apaga arquivos nem comandos — apenas **remove o conteúdo visual** que já apareceu, deixando a tela “limpa”.



Limpar a tela

```
clear
```

O histórico continua existindo (você pode rolar para cima), mas a área visível fica vazia.

Atalho útil:

Em muitos terminais, você pode usar **Ctrl + L** para o mesmo efeito.

Analogia: **clear** é como passar um pano no quadro — você não apaga o que foi escrito, só deixa a superfície limpa para continuar trabalhando.



11. Procurar texto dentro de arquivos — **grep**

O comando **grep** significa **global regular expression print**.

→ literalmente: *“imprimir resultados de uma expressão de busca”*.

Ele serve para **procurar texto dentro de arquivos**, mostrando apenas as linhas que contêm o termo buscado.



Buscar texto em um arquivo

```
grep "texto" arquivo.txt
```

Mostra todas as linhas de `arquivo.txt` que contêm a palavra `"texto"`.

Exemplo de saída:

```
Anderson: lembrete de reunião às 10h
```

✚ Dicas úteis

- Ignorar maiúsculas/minúsculas:

```
grep -i "texto" arquivo.txt
```

- Mostrar número da linha:

```
grep -n "texto" arquivo.txt
```

- Buscar em vários arquivos:

```
grep "erro" *.log
```

- Buscar recursivamente em pastas:

```
grep -r "senha" .
```

Analogia: `grep` funciona como o "Ctrl + F" do terminal — mas muito mais poderoso.

🧩 12. Conceitos de caminhos

Para navegar no terminal, você precisa entender dois tipos de caminhos:

absolutos (completos) e **relativos** (a partir da pasta atual).



Caminho absoluto

Um caminho absoluto começa **na raiz do sistema** (`/`) e mostra **todas** as pastas até o destino.

Exemplos:

```
/home/anderson/projetos  
/c/Users/ander/Documents
```

Características:

- sempre começa com `/`
- funciona de qualquer lugar do sistema
- é o “endereço completo” do arquivo ou pasta

Caminho relativo

Um caminho relativo depende **de onde você está agora** (diretório atual).

Exemplo:

```
cd projetos/meu_app
```

Aqui, `projetos/meu_app` é interpretado **a partir da pasta atual**.

Se você estiver em `/home/anderson`, o comando te levará para:

`/home/anderson/projetos/meu_app`



Símbolos importantes

- `~` → **home** (pasta pessoal do usuário)
- `.` → **diretório atual**
- `..` → **diretório acima**

Analogia:

Caminho absoluto é como um endereço completo com rua e número.

Caminho relativo é como dizer “entre naquela sala ali do lado” — depende de onde você já está.



13. Permissões no Linux (r, w, x)

No Linux, cada arquivo e pasta possui **permissões**, que definem quem pode **ler**, **escrever** ou **executar**.

Exemplo de saída do `ls -l`:

```
-rwxr-xr-- 1 anderson users 1024 Jun 12 09:00 script.sh
```

A primeira parte (`-rwxr-xr--`) representa as permissões.

Ela é dividida assim:

```
[ tipo ][ dono ][ grupo ][ outros ]  
-      rwx      r-x      r--
```



Significado das letras

- **r** → *read* (ler)
- **w** → *write* (escrever)
- **x** → *execute* (executar)

Para arquivos:

- **r** → pode abrir e ler
- **w** → pode editar
- **x** → pode executar (útil para scripts)



Permissões em pastas

Para diretórios, as permissões têm significados diferentes:

- **r** → listar conteúdo da pasta
- **w** → criar, renomear ou apagar arquivos dentro dela
- **x** → entrar na pasta (acessar)

Exemplo prático:

- Uma pasta sem **x** → você *não consegue entrar*, mesmo que tenha **r**
- Uma pasta com **x**, mas sem **r** → você pode entrar, mas *não consegue listar* o que tem dentro

Analogia:

Pense em uma pasta como uma sala:

- **x** é a chave da porta (entrar)
- **r** é poder olhar o que tem dentro
- **w** é poder mexer nos objetos da sala



14. Alterar permissões — **chmod**

O comando **chmod** significa **change mode**

→ literalmente: “mudar o modo (permissões) de um arquivo ou pasta”.

Ele permite adicionar ou remover permissões de **leitura (r)**, **escrita (w)** e **execução (x)**.

Dar permissão de execução

```
chmod +x script.sh
```

Adiciona **x** (executar) ao arquivo.
Necessário para rodar scripts no Linux.

Remover permissão de escrita

```
chmod -w arquivo.txt
```

Remove a permissão de escrita — o arquivo não poderá ser modificado.

Modo numérico (octal)

Cada permissão tem um valor:

```
r = 4  
w = 2  
x = 1
```

Somando esses valores, você define as permissões de:

1. **dono**
2. **grupo**
3. **outros**

Exemplo:

```
chmod 755 arquivo.sh
```

Interpretação:

- **7** → dono: 4 (r) + 2 (w) + 1 (x) = **rwX**
- **5** → grupo: 4 (r) + 1 (x) = **r-X**
- **5** → outros: 4 (r) + 1 (x) = **r-X**

Resultado final:

```
rwXr-Xr-X
```



15. Diferenças entre Linux/Bash, CMD e PowerShell

Ao usar Git Bash no Windows, você está usando um terminal **estilo Linux**, com comandos POSIX. Isso é diferente do **CMD** (linha de comando antiga do Windows) e do **PowerShell** (linha moderna e orientada a objetos).

A tabela abaixo mostra os comandos equivalentes mais comuns:

Ação	Linux / Bash	Windows CMD	PowerShell
Mostrar diretório atual	<code>pwd</code>	<code>cd</code>	<code>Get-Location</code>
Listar arquivos	<code>ls</code>	<code>dir</code>	<code>Get-ChildItem</code> (ou <code>ls</code>)
Mudar de pasta	<code>cd pasta</code>	<code>cd pasta</code>	<code>Set-Location pasta</code> (ou <code>cd</code>)
Criar pasta	<code>mkdir pasta</code>	<code>mkdir pasta</code>	<code>New-Item -ItemType Directory pasta</code>
Criar arquivo	<code>touch arquivo.txt</code>	<code>type nul > arquivo.txt</code>	<code>New-Item arquivo.txt</code>
Copiar arquivo	<code>cp a b</code>	<code>copy a b</code>	<code>Copy-Item a b</code>
Mover/renomear	<code>mv a b</code>	<code>move a b</code>	<code>Move-Item a b</code>
Apagar arquivo	<code>rm a.txt</code>	<code>del a.txt</code>	<code>Remove-Item a.txt</code>
Apagar pasta	<code>rm -r pasta</code>	<code>rmdir /s pasta</code>	<code>Remove-Item pasta -Recurse</code>
Ver conteúdo de arquivo	<code>cat arquivo.txt</code>	<code>type arquivo.txt</code>	<code>Get-Content arquivo.txt</code>
Limpar tela	<code>clear</code>	<code>cls</code>	<code>Clear-Host</code> (ou <code>cls</code>)

Observações importantes

✓ Git Bash aceita comandos Linux

Por isso `ls`, `pwd`, `touch`, `rm` funcionam no Windows **quando você usa Git Bash**.

✓ CMD é mais limitado

É mais antigo, não suporta muitos recursos modernos e não entende comandos POSIX.

✓ PowerShell é poderoso, mas diferente

Ele trabalha com **objetos**, não texto puro.

Por isso seus comandos são mais longos, porém mais robustos.

Quando usar cada um?

- **Git Bash** → ideal para Git, desenvolvimento, scripts simples, comandos Linux.
 - **CMD** → só use se for obrigado (scripts antigos, ferramentas legadas).
 - **PowerShell** → automação avançada, administração de sistemas Windows.
-

Analogia:

Linux/Bash é como dirigir um carro manual — direto e preciso.

CMD é como um carro antigo — funciona, mas limitado.

PowerShell é como um carro moderno automático — mais complexo, mas muito mais poderoso.

Fim do guia

Agora você domina os comandos essenciais para navegar, criar, mover, copiar, apagar e proteger arquivos no terminal — além de entender as diferenças entre Linux/Bash, CMD e PowerShell.

Com esse conhecimento, você está preparado para trabalhar com Linux, Git, automações simples e ambientes de desenvolvimento de forma muito mais segura e consciente.